A VERIFIABLE ENVIRONMENT FOR NEURAL ARCHITECTURE DESIGN

Anonymous authors
Paper under double-blind review

ABSTRACT

Reinforcement learning and agent evaluation both depend on a *verifier*: a function that scores a candidate cheaply and without a human. Code and mathematics have compilers and unit tests; neural architecture design has had no public verifier. We present an environment in which an agent designs a neural network by emitting edits to a typed, structured graph, and a deterministic, sub-millisecond verifier grades the result against structural, budget, and serving-physics constraints, with no human or LLM in the scoring loop. The environment ships (i) a procedural task generator across ten families with satisfiability and non-vacuity proofs, (ii) anti-gaming constraints that reject wholesale rebuilds masquerading as surgical fixes, (iii) a difficulty-calibration harness that reports per-family pass rates against the pass-rate floor labs cite for usable RL environments, and (iv) a grounding study relating the verifier's verdict to real PyTorch behavior. Across 264 architectures, verifier blockers predict forward-pass failure with perfect precision ($96/96$), while graphs the verifier passes construct and train in 90% of cases; we also report, honestly, that the 0–100 health score is a validity margin rather than a quality ranking (within-family Spearman correlation with training progress is weak). Fed into the loop, the verifier is a measurable capability multiplier: on the public benchmark it raises `claude-sonnet-4-6` from 82% to 100% ($+18$ points), a single repair round closing the entire gap, and replicates on a second, weaker model (deepseek-chat, $60\% \rightarrow 88\%$, $+28$); on a curated split of production architectures the same model passes only $7/12$ first try. Using the verifier as ground truth, we also audit an LLM reward model: across eleven reward models, not one approves a blatantly broken design (0% false positive) – yet on *near-miss* designs one edit from correct, the same judges approve 22–48% of provably-broken graphs, failing exactly where reward hacking lives; only the deterministic verifier stays at 0% on both tiers. The environment also trains models: every generated task carries a provably-passing reference, so it mints unlimited verified supervised pairs, and fine-tuning a 1.5B model on 3,000 of them lifts held-out pass@1 from 23% to 86%; RL alone at the same scale is a null we report honestly – the classic SFT-then-RL split, with the environment supplying both stages. Because grading is a pure function, the same environment is at once a leaderboard, an RL training gym, a verified-SFT and reasoning-trace generator, and a ground-truth anchor for LLM reward models; we release all of it.

Reproducibility note. All results marked *measured* are reproduced by the commands in the repository; the calibration, amplification, and reward-model results use provider API keys, the rest need neither keys nor GPUs. Code and data: `https://github.com/neurarch-ai/neurarch-arch-bench` (MIT).

1 INTRODUCTION

The scarce ingredient in modern agent training and evaluation is not the policy but the verifier. SWE-Bench [1] grades a code patch by running the project's own test suite; τ-bench [2] grades a tool-using dialogue by diffing the resulting database state against a goal. Neither requires a human judge or a second model, and this is precisely why they are trusted and why they can be used as training signals.

Neural architecture design has lacked such a verifier, for a structural reason: whether a proposed network is even runnable (attention head counts that divide the embedding dimension, linear widths that match upstream shapes, a graph whose input reaches its output, a parameter count inside a budget) is a pure function only when architectures are represented as typed, structured graphs rather than free-form code. Coding agents emit code, not graphs, so their output carries no shape-level verifiability. We close this gap by building the environment around a typed graph and the deterministic checks that a production editor already runs on it. Existing structured views of architectures, such as Netron [13], are read-only viewers: they render a graph but do not edit it, propagate shapes, or verify, and so cannot serve as an action space or a reward.

Our contributions are: (1) a design-from-spec and edit-in-place environment with a sub-millisecond programmatic verifier; (2) a procedural task generator with satisfiability proofs, non-vacuity proofs, and anti-gaming constraints; (3) a calibration harness that measures difficulty against the labs' usable band, with keyless self-tests that bracket the harness itself; (4) a grounding study connecting verdicts to real PyTorch behavior, reported with its negative results intact; and (5) four empirical results and four released downstream uses of the same verifier: a verifier-in-the-loop lift ($82 \rightarrow 100$, a single repair round), a 62-point held-out lift from fine-tuning on environment-minted verified pairs, an LLM-reward-model audit (a frontier model's 0% false-positive but over-conservative behavior), and a data foundry that mints verified reasoning traces, atop a leaderboard, RL gym, and verified-SFT generator.

2 The environment

State. A typed graph of a neural network: components drawn from a vocabulary of 182 layer types, each carrying parameters, connected by directed edges. **Action.** A batch of structured edits from a fixed vocabulary: `add_component`, `add_connection`, `update_params`, `delete_component`, `replace_model`, and others, the same vocabulary a production editor executes. **Reward.** A grading function returning a 0–100 health score plus a list of hard blockers; Figure 1 shows the full loop.

Formalization. The environment is a deterministic single-agent MDP $(\mathcal{S}, \mathcal{A}, T, R)$. A state $s \in \mathcal{S}$ is a typed graph (V, E): nodes V each carry a type and a parameter dictionary, edges E are directed. An action $a \in \mathcal{A}$ is a batch of typed edits from the fixed vocabulary; the transition $T(s, a)$ applies them, with no environment stochasticity. The reward is the grader's health score $R(s') \in [0, 100]$, and a task's success predicate is $R(s') \geq \tau \wedge \neg\,\mathrm{blocked}(s') \wedge \phi(s')$, where ϕ collects the task's machine-checkable constraints (required types, parameter/KV/latency budgets, action-economy limits). Since T and R are pure functions, an episode is fully reproducible from its seed, and the same R is at once an evaluation metric, an RL reward, and a data-generation filter.

Widths, parameters, and cost. Each node type defines an output width w_{out} and a parameter count from its typed parameters. For the load-bearing types,

$$\text{linear}(d_{\text{in}}, d_{\text{out}}): \quad w_{\text{out}} = d_{\text{out}}, \quad |\theta| = d_{\text{in}}\, d_{\text{out}} + d_{\text{out}}, \tag{1}$$

$$\text{attention}(d, H): \quad w_{\text{out}} = d, \quad |\theta| = 4d^2 + 4d, \tag{2}$$

$$\text{conv}(c_{\text{in}}, c_{\text{out}}, k): \quad w_{\text{out}} = c_{\text{out}}, \quad |\theta| = c_{\text{in}}\, c_{\text{out}}\, k^2 + c_{\text{out}}. \tag{3}$$

Total parameters are $\textsc{Params}(G) = \sum_{v \in V} |\theta(v)|$, and the decode-time KV-cache cost per token is

$$\textsc{Kv}(G) \;=\; \sum_{v \,\in\, \text{attn}(G)} 2\, H_v^{\text{KV}}\, \frac{d_v}{H_v}\, b, \tag{4}$$

for b bytes per cached value (multi-head attention is the case $H_v^{\text{KV}} = H_v$; grouped-query attention shrinks H_v^{KV}). A task with constraints ϕ and threshold τ is solved iff

$$\textsc{Pass}(T, G) \;=\; \big[\, B(G) = \emptyset \,\big] \;\wedge\; \big[\, h(G) \geq \tau \,\big] \;\wedge\; \phi(G), \tag{5}$$

where $B(G)$ is the hard-blocker set of Algorithm 1, $h(G)$ its health score (whose soft-penalty weights are implementation detail: every result in this paper depends only on the success predicate, and the score is diagnostic), and $\phi(G)$ conjoins the machine-checkable constraints: required

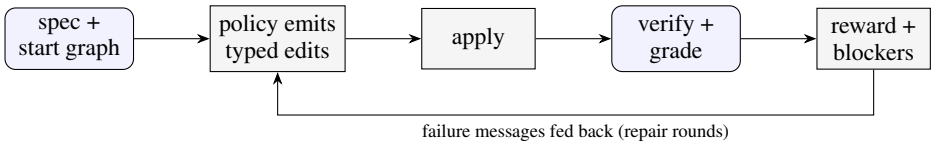

Figure 1: The environment loop. A policy edits a typed graph; a deterministic, sub-millisecond grader returns a reward and hard blockers. Feeding the blocker messages back for repair rounds is the amplification setting (Section 6); with no feedback it is single-shot.

Algorithm 1 GRADE(T, G): the deterministic verifier

Require: task T with constraints ϕ and threshold τ; typed graph $G = (V, E)$
Ensure: health score $h \in [0, 100]$; hard-blocker set B

$B \leftarrow \emptyset$
if $\neg$ REACHES(in(G), out(G)) **then** $B \leftarrow B \cup \{\text{DISCONNECTED}\}$
end if
for each attention node $v \in V$ **do**
  if v.embedDim mod v.numHeads $\neq 0$ **then** $B \leftarrow B \cup \{\text{HEADDIV}(v)\}$
  end if
  if v.numHeads mod v.numKVHeads $\neq 0$ **then** $B \leftarrow B \cup \{\text{GQADIV}(v)\}$
  end if
end for
for each node v with typed input width $w_{\text{in}}(v)$ and parent u **do**
  if $w_{\text{in}}(v) \neq w_{\text{out}}(u)$ **then** $B \leftarrow B \cup \{\text{SHAPEMISMATCH}(v)\}$
  end if
end for
compute PARAMS(G), KV(G), LATENCY(G); add a blocker for each budget in ϕ violated, and for each required type absent
$h \leftarrow 0$ **if** $B \neq \emptyset$ **else** $100 - \sum_{s \in \text{SOFT}(G)} \text{pen}(s)$
return (h, B)

types, a two-sided parameter band $p_{\min} \leq \text{PARAMS}(G) \leq p_{\max}$, KV and latency budgets, and an action-economy limit $|a| \leq a_{\max}$. Every term is a pure function of G, so Eq. (5) is decidable in $O(|V| + |E|)$.

The grader composes three checks, each pure and sub-millisecond: a structural score with hard blockers (disconnected input/output, attention divisibility, parameter bloat), a guardrail pass over the proposed edits, and a shape propagator that catches shape bugs an edit would introduce. No LLM grades anything, so there is no persuasion surface and no stochasticity in the reward.

The checks. The grader runs three families of checks, in order. (i) *Structural blockers*: a disconnected input or output; an attention layer whose `embedDim` is not divisible by `numHeads` (or `numHeads` not divisible by `numKVHeads` under grouped-query attention); a linear layer whose `inFeatures` do not match the upstream width; a parameter count far over budget. (ii) *Serving physics*: parameters, forward multiply-accumulates, and KV-cache bytes per token, computed with the canonical formula (grouped-query attention caches $2 \cdot$ `numKVHeads` $\cdot\, d_{\text{head}} \cdot b$ bytes per token at b bytes per value; multi-head attention is the special case `numKVHeads` $=$ `numHeads`), and checked against per-task parameter, KV, and decode-latency budgets. (iii) *Shape propagation*: a symbolic forward pass over the typed graph that catches interface mismatches an edit would introduce before any tensor is allocated. Each pass is $O(|V| + |E|)$ over the graph and completes in well under a millisecond, which is what makes the reward usable as an inner-loop RL signal rather than an offline evaluation.

A task pairs a natural-language specification with a starting graph and machine-checkable constraints, e.g. *"This encoder fails validation: embedDim (192) is not divisible by numHeads (7). Repair the attention configuration in place with at most 2 actions. Do not rebuild the model from scratch."* Figure 2 shows this state and the blocker the verifier returns.

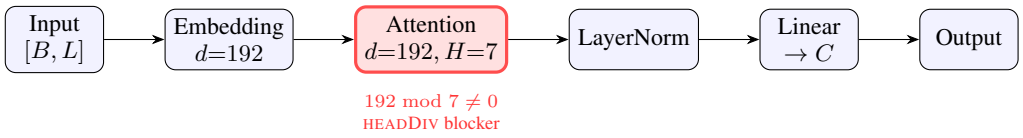

Figure 2: A task state as a typed graph, and a hard blocker the verifier returns in $O(1)$ at the attention node: embedDim=192 is not divisible by numHeads=7 (Algorithm 1, line 4). Because widths and divisibility are typed-node properties, this check is a pure function; on free-form code it would require executing the model. The input carries shape $[B, L]$ (B batch, L sequence length), widening to $[B, L, 192]$ after the embedding. The repair task asks the policy to fix exactly this in ≤ 2 edits without rebuilding.

Failure category (curated split, `claude-sonnet-4-6`)	count
missing required layer type (e.g. attention)	3
disconnected graph (input does not reach output)	2
health score below threshold	1
over parameter budget	1
passed	**7 / 12**

Table 1: First-try failures of a frontier model on the curated production-architecture split. Every failure is detected deterministically by the verifier.

3 TASK GENERATION

Curated tasks are a seed. A deterministic generator mints splits of any size from a seed across ten families: six design-from-spec (dense classifier, autoencoder, convolutional classifier, transformer encoder, grouped-query attention encoder, two-tower retrieval) and four edit-in-place (repair a broken attention configuration, trim an oversized model under a parameter budget, grow an undersized model into a two-sided parameter band, insert normalization). Because tasks are synthesized rather than scraped, a held-out split need never appear on the public web; contamination resistance is a property of construction, not obscurity (we argue this structurally; we have not yet verified it against a web-scale index).

Satisfiability and non-vacuity (measured). Every generated task carries its own reference solution. A 500-case sweep asserts that each reference passes its own task (no unsatisfiable tasks), and every edit-in-place start graph is asserted to fail its own task untouched (no vacuous tasks). Both run in CI without keys.

Anti-gaming. Edit-in-place families forbid `replace_model` and `clear_canvas`, so an agent that cannot diagnose a defect cannot pass by regenerating the whole network; budgets are two-sided bands rather than ceilings, so "delete everything" fails a trim task; a KV-cache budget is always paired with a required-attention constraint, so amputating attention cannot zero the cache. Nine such strategies, their defenses, and the pinned tests that fail if any defense is weakened are enumerated in the repository's red-team report (`ROBUSTNESS.md`). An LLM-driven task proposer accepts nothing without a satisfiability proof: the proposer's own reference must pass the grader under always-enforced safety-net constraints, so LLM authorship, unsafe for human- or LLM-judged benchmarks, is safe here.

A benchmark that bites (measured). On twelve hand-authored curated tasks built from real production architectures (a CIFAR CNN, a DLRM click-through-rate ranker, a Behavior Sequence Transformer, a two-tower retrieval encoder, a semantic-search encoder, and repair/scaling tasks), `claude-sonnet-4-6` passes $7/12$ first try (mean health score 76). The failures are machine-checkable and mundane (Table 1): it omits the required attention layer, leaves the graph disconnected, or blows the parameter budget by an order of magnitude. A benchmark a frontier model clears $12/12$ measures nothing.

Family	single-shot	with verifier feedback
dense classifier	100%	100%
autoencoder	100%	100%
convolutional classifier	100%	100%
GQA encoder	100%	100%
repair attention	100%	100%
trim under budget	100%	100%
grow to band	100%	100%
two-tower retrieval	100%	100%
transformer encoder	**25%**	**100%**
insert normalization	**0%**	**100%**
overall	**82%**	**100%**

Table 2: Per-family pass rate for `claude-sonnet-4-6` on the public benchmark ($n = 12$ per family, 117 graded). Eight families saturate; the two the model finds hard are exactly where verifier-in-the-loop feedback pays off. Overall single-shot 82% (Wilson 95% CI $[74, 88]$) rises to 100% ($[97, 100]$); each hard family moves $12/12$ with feedback ($[76, 100]$).

4 Difficulty calibration

An ungameable environment is still useless at 0% pass (no learning signal) or near 100% (nothing to learn). Practitioners cite a floor near 2–3% pass rate on the hard end (one success per 64–128 rollouts) for a usable RL environment. The calibration harness measures per-family pass rates with Wilson 95% intervals and flags each family's band. Two keyless self-test policies bracket the harness itself: a `reference` policy that replays known-good solutions (measured: 100% pass) and a `noop` policy that submits empty plans (measured: 0% pass). If either self-test deviates, the harness rather than the model is broken, and CI fails.

Measured (`claude-sonnet-4-6`, public benchmark, $n = 12$/family). Single-shot pass rate per family: eight families saturate (100%, a curriculum tier for a frontier model) and two are hard: transformer encoder (25%) and insert-norm (0%) (Table 2). Deterministic grading, reproducible from the seed.

5 Grounding study

Does the verifier's verdict correspond to reality? We generated clean reference architectures and systematically corrupted variants (broken attention divisibility, mismatched linear width, a severed mid-graph edge), built every graph as a PyTorch module, and trained each briefly on a synthetic fixed-teacher task.

Result (measured, 264 graphs, two seeds). Verifier blockers predicted a forward-pass failure with perfect precision: 96 of 96 blocked graphs failed to run. Graphs the verifier passed constructed in 80 of 80 cases and made training progress in 90%. The one systematic miss is honest and instructive: the transparent rubric in this repository does not chase linear widths through the graph, so a width mismatch can pass the rubric and crash PyTorch; the richer verifier in the production system flags this class. We treat the pass/blocked boundary as the grounded claim (Figure 3).

Negative result (measured). Within clean graphs, the 0–100 score's magnitude does *not* rank training quality: Spearman correlation between the score and relative loss decrease was -0.58, because deeper, more structured models make slower per-step progress on the short synthetic probe than tiny ones. The score is therefore a validity margin, not a quality ranking. A learned quality head, trained on (architecture, verdict, real training curve) triples that the environment can mint at scale, is the natural next step and the point at which the verifier would become a calibrated predictor rather than a gate.

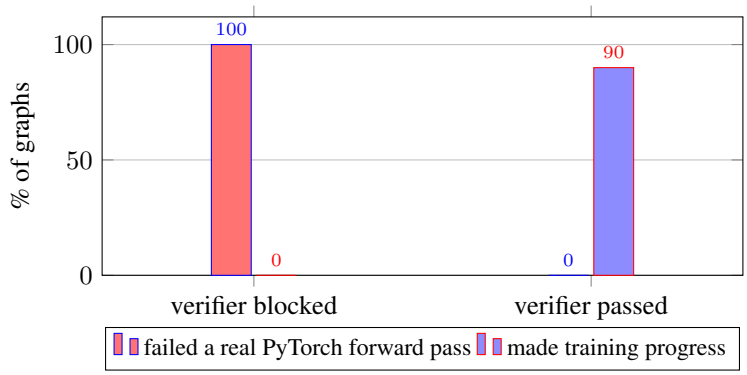

Figure 3: Grounding study (264 graphs, two seeds). Every graph the verifier blocked (96/96) failed a real PyTorch forward pass; graphs it passed constructed (80/80) and made training progress in 90% of cases. The verdict's pass/blocked boundary tracks real behavior.

Algorithm 2 ROLLOUT(T, π, k): verifier-in-the-loop repair

Require: task T; policy π; maximum repair rounds k
Ensure: pass $\in \{0, 1\}$; rounds used

1: $G \leftarrow T$.startGraph; $m \leftarrow \varnothing$ ▷ m: verifier feedback
2: **for** $t \leftarrow 1$ **to** k **do**
3: $a \leftarrow \pi\big(T.\text{spec}, \text{SERIALIZE}(G), m\big)$ ▷ batch of typed edits
4: $G \leftarrow \text{APPLY}(G, a)$
5: $(h, B) \leftarrow \text{GRADE}(T, G)$
6: **if** $B = \emptyset \wedge h \geq \tau \wedge \phi(G)$ **then return** $(1, t)$
7: **end if**
8: $m \leftarrow \text{EXPLAIN}(B)$ ▷ the verifier's failure messages
9: **end for**
10: **return** $(0, k)$

6 VERIFIER-IN-THE-LOOP AMPLIFICATION

Single-shot is the k=1 special case (no feedback consumed); the amplification setting sets k=3. Algorithm 2 states the loop; the only difference between the two arms is whether line 8's feedback m re-enters line 3. *Pending a run.* Because grading is free, the verifier's failure messages can be fed back to a policy for repair rounds. We measure, per provider, the pass rate of a single shot versus the pass rate when up to k repair rounds are allowed, holding tasks, model, and prompt fixed so that the only variable is the feedback loop. The framing is deliberately pro-model: the environment's value is the lift it adds to any model.

Model	single-shot pass	with verifier feedback	lift
claude-sonnet-4-6	82%	100%	+18 pts
deepseek-chat (V3)	60%	88%	+28 pts

Table 3: Verifier-in-the-loop lift, per provider. Same tasks, model, and prompt; the only variable is the feedback loop.

The lift is entirely on the two families a frontier model finds hard (Figure 4): insert-norm 0% $\rightarrow$ 100% and transformer encoder 25% $\rightarrow$ 100% once the verifier's failure messages are fed back, while the eight curriculum families are already at 100%. A single repair round closes the whole gap (Table 4). $n = 12$ per family, 117 graded (three transient network errors excluded). The effect is not model-specific: on the same tasks, a second, weaker model (deepseek-chat) goes 60% $\rightarrow$ 88% (+28 points, $n = 60$), with every fix again landing at k=2 and none at k=3. The weaker the model, the more the verifier is worth.

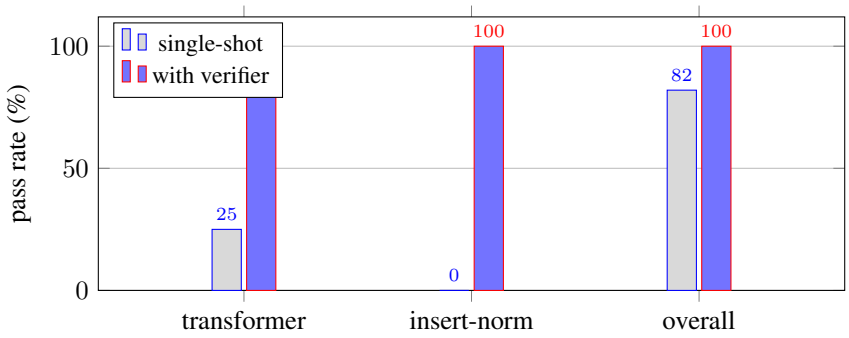

Figure 4: Verifier-in-the-loop lift for `claude-sonnet-4-6`. The overall gain ($82 \rightarrow 100$) lands entirely on the two families a frontier model finds hard; the eight saturated families (all at 100%) are omitted for clarity.

max repair rounds k	claude-sonnet-4-6	deepseek-chat
$k = 1$ (single-shot)	82% $[74, 88]$	60% $[47, 71]$
$k = 2$	$\mathbf{100}\%$ $[97, 100]$	$\mathbf{88}\%$ $[78, 94]$
$k = 3$	100% $[97, 100]$	88% $[78, 94]$

Table 4: Ablation on the number of verifier-feedback repair rounds, two models (claude n=117, deepseek n=60; Wilson intervals). For *both*, a single repair round (k=2) provides the entire lift – every task the verifier can help fix is fixed after one round of its failure messages – and k=3 adds nothing. The pattern is not model-specific.

Numbers are over the tasks graded before an API-credit cutout ($n \approx 8$–10 per family, 95 graded).

7 Training on the environment

The environment trains models through two channels, and we report both honestly.

Verified SFT: the data foundry works. Every generated task carries a reference solution that provably passes the grader, so the environment mints unlimited verified (spec $\rightarrow$ edits) supervised pairs, keylessly and for free. Fine-tuning Qwen2.5-1.5B-Instruct on 3,000 such pairs (LoRA, one free T4, under ten minutes) transforms held-out performance (Table 5): pass@1 rises from 23.4% (Wilson $[15, 35]$) to $\mathbf{85.9}\%$ ($[75, 92]$), a $+62$-point lift with non-overlapping intervals; parse failures collapse from $19/64$ to $2/64$; mean reward rises from 0.35 to 1.19. The eval split (seed 999) is generated from a seed disjoint from the training data's, so the gain is generalization to fresh instances of the ten families, not recall of seen tasks. The surviving failures are substantive design errors the verifier pinpoints (disconnected graphs, too few components), no longer formatting. Data scaling is steep and non-monotonic at the low end (Figure 5): 250 and 500 pairs *underperform* the raw instruct model (9.4% and 14.1% vs. 25.0%) – a half-learned edit format displaces the base model's occasional valid outputs before replacing them – while 1,000 pairs recover to 34.4% and 3,000 reach 85.9%. The foundry mints pairs for free, so the operative cost of the full lift is minutes of generation plus minutes of fine-tuning. The lift itself replicates: an independent rerun in a fresh session lands at 79.7% ($51/64$, Wilson $[68, 88]$). Transfer, however, is in-distribution: on the twelve hand-authored curated tasks the SFT model is unchanged ($3/12$ before and after), so the gain reflects mastery of the generated families rather than general design skill – broadening the diversity of the minted data is the obvious next lever, and the generator makes that a parameter rather than a labeling effort.

RL alone at small scale: an honest null, now explained. The released GRPO [6] loop runs end-to-end against the reward server on the same hardware, but RL *from the raw instruct model* does not move held-out metrics: two runs (learning rates $10^{-6} \times 150$ and $10^{-5} \times 250$ steps, every checkpoint evaluated and reported) sit at or below their own baselines. The SFT result explains the failure: the raw policy cannot emit parseable edits on a third of tasks, so the group-relative policy gradient

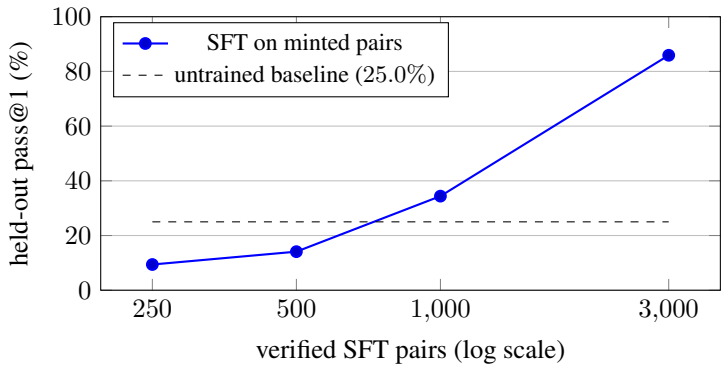

Figure 5: Held-out pass@1 vs. number of environment-minted verified SFT pairs (Qwen2.5-1.5B, LoRA, 2 epochs, seed-999 split, n=64). The curve is steep: below ∼1k pairs the half-learned format underperforms the raw instruct model; 3k pairs reach 85.9%. Pairs are free to mint, so the x-axis costs seconds of generation.

	pass@1	parse failures	mean reward
Qwen2.5-1.5B (untrained)	23.4% [15, 35]	19/64	0.35
+ SFT on 3k verified pairs	**85.9**% [75, 92]	**2/64**	**1.19**

Table 5: Training on environment-minted data (LoRA, one free T4; held-out seed-999 split, n=64, identical eval configuration). Supervised fine-tuning on 3,000 verified pairs the generator mints for free lifts pass@1 by 62 points with non-overlapping Wilson intervals and nearly eliminates parse failures. GRPO from the raw model, in contrast, is a null at this scale (see text).

starves. This is the classic SFT-then-RL split – supervised learning teaches the action format, RL then refines – and the environment supplies both stages: the verified SFT data and the deterministic reward. Scaled SFT-then-RL is the natural next experiment.

8 Auditing LLM reward models with the verifier

Frontier labs increasingly use a strong model *as* a reward model to optimize objectives that are not directly verifiable (xAI reported this for Grok 4.1), a practice studied more broadly as LLM-as-a-judge [12]. The known failure mode is that an LLM reward model drifts: it approves answers that are actually broken. In a domain with a verifiable reward that drift is *measurable*. `reward_anchor.mjs` builds a verifier-labeled set (each design provably passes or fails: a reference design passes, its unsolved starting graph fails) and, given an LLM acting as a reward model, reports its agreement with the verifier and, crucially, its **false-positive rate**: how often it approves a design the verifier proves is broken. Across eleven reward models spanning closed-frontier (Claude, Grok) and open-weights (Qwen, Mistral, DeepSeek, Gemma, Llama), the false-positive rate is uniformly 0%: not one approves a design the verifier proves broken. The failure mode that corrupts RLVR – rewarding a broken design – does not appear; the only errors are false *negatives* (0–10%, over-conservative), which waste valid designs but do not poison the reward. An LLM reward model here is therefore safe but lossy, and the verifier does the same job with 0% false negatives, deterministically and for free: a verdict costs microseconds of CPU, versus a multi-second, paid, stochastic API round-trip per judgment. This looks reassuring – but the negatives so far are blatant (an unsolved starting graph).

The 0% does not survive near-misses. We re-run the audit with *near-miss* negatives: passing designs corrupted by a single edit (a dropped connection, an attention head count bumped off divisibility, a dropped component), each re-graded to confirm it fails. On these, the same judges collapse (Table 6): Claude approves 30% of provably-broken designs (Wilson [20, 43]), Grok 22% [13, 34], and Qwen-2.5-72B – the one model with *perfect* agreement on blatant negatives – approves 48% [36, 61] while rejecting nothing, i.e. it defaults to approval whenever a graph merely looks plausible.

reward model	blatant FP	near-miss FP	near-miss FN
qwen-2.5-72b-instruct	0.0%	**48.3**% [36, 61]	0.0%
claude-sonnet-4-6	0.0%	30.0% [20, 43]	6.7%
grok-4	0.0%	21.7% [13, 34]	5.0%
deterministic verifier (ours)	0%	**0**%	**0**%

Table 6: The same audit with near-miss negatives: passing designs one edit from correct (n=60 each; corruption mix: dropped connection, broken head divisibility, dropped component, each verified to fail). The 0% false-positive rate on blatant negatives collapses to 22–48%; the model that was perfect on blatant negatives (Qwen) is the worst here, approving whatever looks plausible. LLM judges fail precisely where reward hacking lives; the verifier does not.

model	tier	agreement	false-pos.	false-neg.
qwen-2.5-72b-instruct	open	100.0%	**0.0**%	0.0%
grok-4.5	frontier	95.0%	**0.0**%	5.0%
claude-sonnet-4-6	frontier	93.3%	**0.0**%	6.7%
mistral-large	open	93.3%	**0.0**%	6.7%
deepseek-chat (V3)	open	93.3%	**0.0**%	6.7%
grok-4.20-reasoning	frontier	93.3%	**0.0**%	6.7%
grok-4	frontier	91.7%	**0.0**%	8.3%
grok-4-fast	frontier	91.7%	**0.0**%	8.3%
gemma-2-27b-it	open	90.0%	**0.0**%	10.0%
llama-3.3-70b-instruct	open	90.0%	**0.0**%	10.0%
deepseek-r1 (reasoning)	open	90.0%	**0.0**%	10.0%
deterministic verifier (ours)	—	100%	**0**%	**0**%

Table 7: Eleven LLM reward models ($n = 60$ each) spanning closed-frontier (Claude, Grok) and open-weights (Qwen, Mistral, DeepSeek, Gemma, Llama), including two reasoning models (Grok, DeepSeek-R1), each judging a 60-example labeled set (30 valid, 30 broken). False positive: approving a design the verifier proves broken; false negative: rejecting a valid one. *Not one* of the eleven has a false positive (0% across all); the only failure mode is over-conservatism (0–10% false negative). The verifier matches the best model ($0/0$) at zero cost and deterministically. Section text and Table 6 show this 0% does not survive subtler, near-miss negatives.

The inversion matters: LLM reward models are trustworthy exactly when errors are obvious, and fail exactly where reward hacking lives – subtle defects one edit from correct. The deterministic verifier is 0% on both difficulty tiers by construction. This is a use of the environment beyond training or evaluation: a ground-truth anchor exposing when the LLM reward models labs deploy can and cannot be trusted.

9 Verified reasoning traces

The same verifier is a data foundry for the input reasoning-model post-training consumes: (spec → reasoning → design) triples where the design is re-graded and only passing traces are kept (rejection sampling). No trace enters the set unless its design provably solves the spec. A frontier model's verified yield under this filter – the fraction of its reasoned designs that pass – was 72.5% (327 of 451 tasks, with 49 API failures excluded) on a 500-task run, producing 327 verified traces; the released tool records the yield per run and scales to larger sets. The 327-trace set is public: https://huggingface.co/datasets/neurarch-ai/arch-reasoning-claude. Because the split is seed-addressable, a private seed produces reasoning data whose designs never appeared on the public web.

10 Related work

Verifiable agent benchmarks. SWE-Bench [1] and τ-bench [2] established the programmatic-verifier pattern in code and tool use; SWE-Gym [3] showed that a few thousand verifiable tasks

	domain	verifiable reward	procedural, contam.-resist.	anti-gaming red-teamed	typed-graph action space
SWE-Bench [1]	code	✓	partial	–	–
τ-bench [2]	tool use	✓	–	–	–
SWE-Gym [3]	code	✓	partial	–	–
classical NAS [11]	architecture	proxy	–	–	–
ours	**architecture**	✓	✓	✓	✓

Table 8: Where this environment sits: architecture design as a verifiable-reward domain, with a contamination-resistant procedural generator, red-teamed anti-gaming defenses, and a typed-graph action space.

suffice to train agents to a double-digit absolute improvement on SWE-Bench, and did so as free, open research. We target the same pattern in a domain that lacked a public verifier (Table 8).

RL environments and verifiable rewards. Training on verifiable rewards (RLVR) is now central to reasoning-model post-training [4; 5], typically with policy-gradient methods such as PPO [7] and GRPO [6], preference optimization [9], or rejection sampling in the STaR [8] style. A growing market supplies labs with RL environments; reward-hacking resistance is the quality bar most consistently cited, and usable environments are expected to expose a hard band near a 2–3% pass floor. We build directly against both criteria, and ship GRPO and STaR loops against the same grader.

Neural architecture search. Classical NAS [11] automates architecture design under a proxy objective; our environment differs in providing a human-legible, per-edit, sub-millisecond verifier over a typed graph, and in serving as a training and evaluation substrate rather than a single search procedure. The accompanying product exposes a deterministic, constraint-satisfying design search (given a parameter budget, KV-cache budget, and target GPU, it returns the Pareto front over capacity, parameters, and KV cache), a NAS-lite special case built entirely from the verifier's physics rather than a learned proxy objective. Differentiable and one-shot NAS [10] make gradient search tractable by relaxing a small, hard-coded cell vocabulary and optimizing task loss; structural legality is assumed, not checked. A per-edit legality verifier is orthogonal: it can veto the candidates of *any* search procedure – a DARTS derivation, an evolutionary mutation, or an LLM agent's edits – before GPU time is spent, so it composes with these methods rather than competing with them.

11 Limitations

The transparent rubric released here does not verify linear-width consistency (Section 5); the health-score magnitude is not a quality ranking (Section 5); the reward path does not execute PyTorch per rollout, so shape-class failures are caught statistically rather than per-instance; and generated references lean on `replace_model` for design-from-spec families, biasing imitation style. Each is documented with its measurement or its planned mitigation in `ROBUSTNESS.md`.

12 Conclusion

Representing architectures as typed graphs turns "is this network sound, and what will it cost to serve" into a pure function. That function is simultaneously a leaderboard, an RL reward, and a verified-data generator, and it grounds against real PyTorch behavior on the claim that matters. We release the environment, the generator, the calibration and red-team harnesses, and the training loops, so that architecture design can join code and mathematics as a domain with a public verifier.

Broader Impact

The environment lowers the cost of designing valid, servable architectures and of training and evaluating agents that do so, which should broaden access beyond teams with dedicated ML-research staff. Two risks warrant note. First, a verifier defines what "good" means; ours checks structural validity and serving cost, not fairness, safety, or data appropriateness, and must not be mistaken for a quality oracle – the negative result in Section 5 is deliberate. Second, verified-data generation can

accelerate capability gains; we mitigate by releasing the methodology openly, so the same tools that build also audit (the reward-model analysis above). The benchmark contains no personal data and no scraped content: every task is synthetic or hand-authored.

REPRODUCIBILITY

```
git clone https://github.com/neurarch-ai/neurarch-arch-bench
cd neurarch-arch-bench
npx vitest run                                  # satisfiability + non-vacuity + anti-gamin
node calibrate.mjs --policy=reference           # harness self-test (must be 100%)
node calibrate.mjs --policy=noop                # harness self-test (must be 0%)
node training/build_sft_dataset.mjs \
     --count=3000 --seed=20260713          # mint verified SFT pairs (no key)
python training/train_sft.py \
     --data arch-design-sft.chat.jsonl  # 23.4% -> 85.9% (one T4, <10 min)
python training/train_grpo.py --eval-only --seed 999 --count 64 \
     --model out/sft-arch/checkpoint-final
node dump_grounding_set.mjs --count=40 --seed=123 --out=g.jsonl
python training/grounding_at_scale.py --set g.jsonl
python training/analyze_grounding.py
```

A EXAMPLE CURATED TASKS

The twelve curated tasks are hand-authored from real production architectures. Figure 6 sketches four, as the typed graphs the environment operates on; each is a distinct family (vision, ranking, sequence, retrieval) with its own required layer types and budgets.

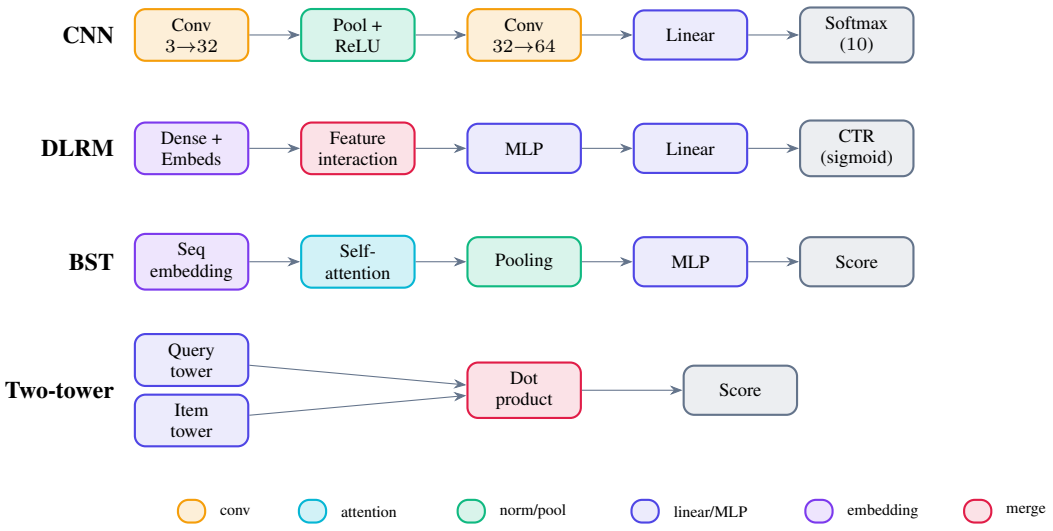

Figure 6: Four of the twelve curated tasks, as typed graphs, coloured by layer type (the same convention the Neurarch canvas uses): a CIFAR-10 CNN, a DLRM click-through-rate ranker, a Behavior Sequence Transformer (which *requires* an attention layer), and a two-tower retrieval model (two encoders scored by a dot product). These are the states an agent edits and the verifier grades.

REFERENCES

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024.

[2] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv:2406.12045*, 2024.

[3] J. Pan, X. Wang, G. Neubig, et al. Training Software Engineering Agents and Verifiers with SWE-Gym. *arXiv:2412.21139*, 2024.

[4] N. Lambert et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv:2411.15124*, 2024.

[5] DeepSeek-AI (D. Guo et al.). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*, 2025.

[6] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, 2024.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.

[8] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. STaR: Bootstrapping Reasoning with Reasoning. *NeurIPS*, 2022.

[9] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct Preference Optimization. *NeurIPS*, 2023.

[10] H. Liu, K. Simonyan, and Y. Yang. DARTS: Differentiable Architecture Search. *ICLR*, 2019.

[11] T. Elsken, J. H. Metzen, and F. Hutter. Neural Architecture Search: A Survey. *JMLR*, 20(55):1–21, 2019.

[12] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023.

[13] L. Roeder. Netron: Visualizer for neural network, deep learning and machine learning models. Software, https://github.com/lutzroeder/netron.